VenueVista

Complete project source and configuration listing

55 project files

.env.example

```
# Copy to .env and replace these placeholders with local values.
DB_URL=jdbc:oracle:thin:@//localhost:1521/XEPDB1
DB_USERNAME=your_oracle_username
DB_PASSWORD=replace_with_your_oracle_password
JWT_SECRET=replace_with_a_random_secret_at_least_32_bytes_long
JWT_EXPIRATION_MS=86400000
CORS_ORIGIN=http://localhost:4200
```

.gitignore

```
■# Java / Maven
backend/target/
*.class
*.jar

# Node / Angular
frontend/node_modules/
frontend/.angular/
frontend/dist/
npm-debug.log*

# Local files and generated scratch data
.env
!.env.example
*.log
tmp/
.DS_Store
Thumbs.db

# Local IDE settings and OS credentials
.idea/
.vscode/
*.pem
*.key
*.p12
*.jks
```

backend/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:
schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>
  <groupId>com.venuevista</groupId>
  <artifactId>venuevista-api</artifactId>
  <version>1.0.0</version>
  <properties>
    <java.version>17</java.version>
    <lombok.version>1.18.48</lombok.version>
  </properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></
dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></
dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></
dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></
dependency>
    <dependency><groupId>com.oracle.database.jdbc</groupId><artifactId>ojdbc11</artifactId><scope>runtime</scope></
dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.5</version></
dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>0.12.5</
version><scope>runtime</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>0.12.5</
version><scope>runtime</scope></dependency>
    <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optional></
dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</
artifactId><scope>test</scope></dependency>
  </dependencies>
  <build><plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <path><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><version>${lombok.version}</
version></path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId>
      <configuration><excludes><exclude><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId></
exclude></excludes></configuration>
    </plugin>
  </plugins></build>
</project>
```

backend/src/main/java/com/venuevista/model/AppUser.java

```
package com.venuevista.model;
import jakarta.persistence.*;
import lombok.*;
@Entity @Table(name = "APP_USERS") @Getter @Setter @NoArgsConstructor
public class AppUser {
  public enum Role { ADMIN, ORGANIZER, CUSTOMER }
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
  @Column(nullable = false) private String name;
  @Column(nullable = false, unique = true) private String email;
  @Column(nullable = false) private String passwordHash;
  @Enumerated(EnumType.STRING) private Role role = Role.CUSTOMER;
}
```

backend/src/main/java/com/venuevista/model/BookingOrder.java

```
package com.venuevista.model;
import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.*;
@Entity @Table(name = "BOOKING_ORDERS") @Getter @Setter @NoArgsConstructor
public class BookingOrder {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
  @Column(unique = true) private String code;
  @ManyToOne(fetch = FetchType.LAZY, optional = false) private AppUser user;
  @ManyToOne(fetch = FetchType.LAZY, optional = false) private Event event;
  @Column(precision = 12, scale = 2) private BigDecimal total;
  private LocalDateTime createdAt = LocalDateTime.now();
  @OneToMany(mappedBy = "order") private List<Ticket> tickets = new ArrayList<>();
}
```

backend/src/main/java/com/venuevista/model/Event.java

```
package com.venuevista.model;
import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
@Entity @Table(name = "EVENTS") @Getter @Setter @NoArgsConstructor
public class Event {
  public enum Status { ACTIVE, CANCELLED }
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
  @Column(nullable = false) private String title;
  private String category;
  @Column(length = 2000) private String description;
  @Column(nullable = false) private LocalDateTime startTime;
  @ManyToOne(fetch = FetchType.LAZY, optional = false) private Venue venue;
  @Column(precision = 10, scale = 2) private BigDecimal basePrice;
  private String imageUrl;
  @Enumerated(EnumType.STRING) private Status status = Status.ACTIVE;
}
```

backend/src/main/java/com/venuevista/model/Ticket.java

```
package com.venuevista.model;
import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
@Entity @Table(name = "TICKETS") @Getter @Setter @NoArgsConstructor
public class Ticket {
  public enum Tier { STANDARD, PREMIUM, VIP }
  public enum Status { AVAILABLE, SOLD }
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
  @ManyToOne(fetch = FetchType.LAZY, optional = false) private Event event;
  private String rowLabel;
  private int seatNumber;
  @Enumerated(EnumType.STRING) private Tier tier;
  @Column(precision = 10, scale = 2) private BigDecimal price;
  @Enumerated(EnumType.STRING) private Status status = Status.AVAILABLE;
  @ManyToOne(fetch = FetchType.LAZY) private BookingOrder order;
  public String label() { return rowLabel + seatNumber; }
}
```

backend/src/main/java/com/venuevista/model/Venue.java

```
package com.venuevista.model;
import jakarta.persistence.*;
import lombok.*;
@Entity @Table(name = "VENUES") @Getter @Setter @NoArgsConstructor
public class Venue {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
  @Column(nullable = false) private String name;
  private String city;
  private String address;
  private int rowsCount;      // seating chart: number of rows (A, B, C ...)
  private int seatsPerRow;    // seating chart: seats in each row
  @Transient public int getCapacity() { return rowsCount * seatsPerRow; }
}
```

backend/src/main/java/com/venuevista/repo/EventRepository.java

```
package com.venuevista.repo;
import com.venuevista.model.Event;
import org.springframework.data.jpa.repository.*;
import java.util.List;
public interface EventRepository extends JpaRepository<Event, Long> {
  @Query("select e from Event e join fetch e.venue order by e.startTime")
  List<Event> findAllWithVenue();
  long countByVenueId(Long venueId);
}
```

backend/src/main/java/com/venuevista/repo/OrderRepository.java

```
package com.venuevista.repo;
import com.venuevista.model.BookingOrder;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
public interface OrderRepository extends JpaRepository<BookingOrder, Long> {
  List<BookingOrder> findByUserIdOrderByCreatedAtDesc(Long userId);
}
```

backend/src/main/java/com/venuevista/repo/TicketRepository.java

```
package com.venuevista.repo;
import com.venuevista.model.Ticket;
import jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import java.util.List;
public interface TicketRepository extends JpaRepository<Ticket, Long> {
  List<Ticket> findByEventIdOrderByRowLabelAscSeatNumberAsc(Long eventId);
  @Lock(LockModeType.PESSIMISTIC_WRITE)
  @Query("select t from Ticket t where t.id in :ids")
  List<Ticket> lockAll(@Param("ids") List<Long> ids);
  @Query("select t.event.id, t.status, count(t) from Ticket t group by t.event.id, t.status")
  List<Object[]> countsByEvent();
  @Query("select t from Ticket t join fetch t.event where t.status = :s")
  List<Ticket> withStatus(@Param("s") Ticket.Status s);
}
```

backend/src/main/java/com/venuevista/repo/UserRepository.java

```
package com.venuevista.repo;
import com.venuevista.model.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
public interface UserRepository extends JpaRepository<AppUser, Long> {
  Optional<AppUser> findByEmail(String email);
  boolean existsByEmail(String email);
}
```

backend/src/main/java/com/venuevista/repo/VenueRepository.java

```
package com.venuevista.repo;
import com.venuevista.model.Venue;
import org.springframework.data.jpa.repository.JpaRepository;
public interface VenueRepository extends JpaRepository<Venue, Long> {}
```

backend/src/main/java/com/venuevista/security/JwtFilter.java

```
package com.venuevista.security;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
import java.util.List;
@Component @RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {
  private final JwtUtil jwt;
  @Override
  protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
 FilterChain chain) throws ServletException, IOException {
    String h = req.getHeader("Authorization");
    if (h != null && h.startsWith("Bearer ")) {
      try {
        Claims c = jwt.parse(h.substring(7));
        var auth = new UsernamePasswordAuthenticationToken(c.getSubject(), null,
          List.of(new SimpleGrantedAuthority("ROLE_" + c.get("role", String.class))));
        SecurityContextHolder.getContext().setAuthentication(auth);
      } catch (JwtException | IllegalArgumentException ignored) { /* invalid token -> stays anonymous */ }
    }
    chain.doFilter(req, res);
  }
}
```

backend/src/main/java/com/venuevista/security/JwtUtil.java

```
package com.venuevista.security;
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import javax.crypto.SecretKey;
import java.util.Date;
@Component
public class JwtUtil {
  private final SecretKey key;
  private final long expMs;
  public JwtUtil(@Value("${app.jwt.secret}") String secret, @Value("${app.jwt.expiration-ms}") long expMs) {
    this.key = Keys.hmacShaKeyFor(secret.getBytes());
    this.expMs = expMs;
  }
  public String generate(String email, String role) {
    return Jwts.builder().subject(email).claim("role", role).issuedAt(new Date())
      .expiration(new Date(System.currentTimeMillis() + expMs)).signWith(key).compact();
  }
  public Claims parse(String token) { return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).
getPayload(); }
}
```

backend/src/main/java/com/venuevista/security/SecurityConfig.java

```
package com.venuevista.security;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.*;
import java.util.List;
@Configuration @RequiredArgsConstructor
public class SecurityConfig {
  private final JwtFilter jwtFilter;
  @Value("${app.cors-origin}") private String corsOrigin;

  @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

  @Bean SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.cors(Customizer.withDefaults()).csrf(c -> c.disable())
      .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .exceptionHandling(e -> e.authenticationEntryPoint((rq, rs, ex) -> rs.sendError(401, "Please sign in")))
      .authorizeHttpRequests(a -> a
        .requestMatchers("/api/auth/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/events/**", "/api/venues/**").permitAll()
        .requestMatchers("/api/analytics/**").hasAnyRole("ADMIN", "ORGANIZER")
        .requestMatchers("/api/events/**", "/api/venues/**").hasAnyRole("ADMIN", "ORGANIZER")
        .anyRequest().authenticated())
      .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
  }

  @Bean CorsConfigurationSource corsSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(List.of(corsOrigin));
    c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    c.setAllowedHeaders(List.of("*"));
    UrlBasedCorsConfigurationSource s = new UrlBasedCorsConfigurationSource();
    s.registerCorsConfiguration("/**", c);
    return s;
  }
}
```

backend/src/main/java/com/venuevista/service/AnalyticsService.java

```
package com.venuevista.service;
import com.venuevista.model.*;
import com.venuevista.repo.*;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class AnalyticsService {
  private final TicketRepository tickets;
  private final OrderRepository orders;
  private final EventRepository events;

  public record Kpis(BigDecimal revenue, long ticketsSold, long totalSeats, double occupancy, long events,
 BigDecimal avgOrder) {}
  public record EventRevenue(String title, BigDecimal revenue, long sold, long capacity) {}
  public record Daily(LocalDate date, BigDecimal revenue, long orders) {}
  public record Summary(Kpis kpis, List<EventRevenue> topEvents, List<Daily> daily, Map<String, Long> soldByTier) {}

  public Summary summary() {
    List<Ticket> sold = tickets.withStatus(Ticket.Status.SOLD);
    long total = tickets.count();
    BigDecimal revenue = sold.stream().map(Ticket::getPrice).reduce(BigDecimal.ZERO, BigDecimal::add);
    List<BookingOrder> all = orders.findAll();

    Map<Long, long[]> cap = new HashMap<>();
    for (Object[] r : tickets.countsByEvent()) cap.computeIfAbsent((Long) r[0], k -> new long[1])[0] += (Long) r[2];

    List<EventRevenue> top = sold.stream().collect(Collectors.groupingBy(t -> t.getEvent())).entrySet().stream()
      .map(en -> new EventRevenue(en.getKey().getTitle(),
        en.getValue().stream().map(Ticket::getPrice).reduce(BigDecimal.ZERO, BigDecimal::add),
        en.getValue().size(), cap.getOrDefault(en.getKey().getId(), new long[1])[0]))
      .sorted(Comparator.comparing(EventRevenue::revenue).reversed()).limit(6).toList();

    LocalDate today = LocalDate.now();
    List<Daily> daily = new ArrayList<>();
    for (int i = 13; i >= 0; i--) {
      LocalDate d = today.minusDays(i);
      List<BookingOrder> day = all.stream().filter(o -> o.getCreatedAt().toLocalDate().equals(d)).toList();
      daily.add(new Daily(d, day.stream().map(BookingOrder::getTotal).reduce(BigDecimal.ZERO, BigDecimal::add), day.
size()));
    }
    Map<String, Long> byTier = new LinkedHashMap<>();
    for (Ticket.Tier t : Ticket.Tier.values()) byTier.put(t.name(), sold.stream().filter(x -> x.getTier() == t).
count());

    BigDecimal avg = all.isEmpty() ? BigDecimal.ZERO : revenue.divide(BigDecimal.valueOf(all.size()), 2, java.math.
RoundingMode.HALF_UP);
    double occ = total == 0 ? 0 : Math.round(sold.size() * 1000.0 / total) / 10.0;
    return new Summary(new Kpis(revenue, sold.size(), total, occ, events.count(), avg), top, daily, byTier);
  }
}
```

backend/src/main/java/com/venuevista/service/DataSeeder.java

```
package com.venuevista.service;
import com.venuevista.model.*;
import com.venuevista.repo.*;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.*;
/** Seeds demo accounts, venues, events and a fortnight of sales so the dashboard is alive on first run. */
@Component @RequiredArgsConstructor
public class DataSeeder implements CommandLineRunner {
  private final UserRepository users; private final VenueRepository venues; private final EventRepository events;
  private final TicketRepository tickets; private final OrderRepository orders;
  private final EventService eventService; private final PasswordEncoder enc;

  private AppUser user(String name, String email, String pw, AppUser.Role role) {
    AppUser u = new AppUser(); u.setName(name); u.setEmail(email); u.setPasswordHash(enc.encode(pw)); u.setRole(role);
 return users.save(u);
  }
  private Venue venue(String n, String city, String addr, int rows, int seats) {
    Venue v = new Venue(); v.setName(n); v.setCity(city); v.setAddress(addr); v.setRowsCount(rows); v.
setSeatsPerRow(seats); return venues.save(v);
  }

  @Override @Transactional
  public void run(String... args) {
    if (users.count() > 0) return;
    user("Asha Admin", "admin@venuevista.com", "Admin@123", AppUser.Role.ADMIN);
    user("Omar Organizer", "organizer@venuevista.com", "Organizer@123", AppUser.Role.ORGANIZER);
    List<AppUser> fans = List.of(
      user("Riya Sharma", "customer@venuevista.com", "Customer@123", AppUser.Role.CUSTOMER),
      user("Kabir Malhotra", "kabir@example.com", "Customer@123", AppUser.Role.CUSTOMER),
      user("Meera Iyer", "meera@example.com", "Customer@123", AppUser.Role.CUSTOMER));

    Venue grand = venue("Kamani Auditorium", "New Delhi", "1 Copernicus Marg", 12, 20);
    Venue arena = venue("Jawaharlal Nehru Indoor Arena", "New Delhi", "Lodhi Road", 16, 28);
    Venue club  = venue("Blue Room Live", "Gurugram", "Cyber Hub, DLF Phase 2", 8, 14);

    Object[][] seed = {
      {"Midnight Raga: A Sitar & Strings Evening", "Concert",
 "An immersive night of classical sitar with a live string ensemble.", 6, 20, grand, 1200},
      {"Delhi Comedy Carnival", "Comedy", "Six of the country's sharpest stand-ups on one stage.", 10, 20, club, 800},
      {"The Last Metro: A Stage Play", "Theatre", "A gripping original drama set on the final train home.", 14, 19,
 grand, 650},
      {"Neon Pulse Festival Night", "Festival", "Electronic headliners, laser stage and a 4-hour closing set.", 21,
 20, arena, 1800},
      {"Premier Basketball Finals — Game 3", "Sports", "The series is level. Game 3 decides momentum.", 27, 19, arena,
 1500},
      {"Indie Unplugged Sessions", "Concert", "Intimate acoustic sets from three emerging bands.", 3, 20, club, 500},
    };
    Random rnd = new Random(7);
    for (Object[] s : seed) {
      Event e = new Event(); e.setTitle((String) s[0]); e.setCategory((String) s[1]); e.setDescription((String) s[2]);
      e.setStartTime(LocalDateTime.now().plusDays((Integer) s[3]).withHour((Integer) s[4]).withMinute(0).
withSecond(0).withNano(0));
      e.setVenue((Venue) s[5]); e.setBasePrice(BigDecimal.valueOf((Integer) s[6])); events.save(e);
      eventService.createSeats(e, (Venue) s[5]);

      List<Ticket> all = tickets.findByEventIdOrderByRowLabelAscSeatNumberAsc(e.getId());
      Collections.shuffle(all, rnd);
      int target = (int) (all.size() * (0.3 + rnd.nextDouble() * 0.4));
      for (int i = 0; i < target; ) {
        int n = Math.min(1 + rnd.nextInt(4), target - i);
        List<Ticket> group = all.subList(i, i + n);
        BookingOrder o = new BookingOrder();
        o.setUser(fans.get(rnd.nextInt(fans.size()))); o.setEvent(e);
        o.setCode("VV-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase());
        o.setCreatedAt(LocalDateTime.now().minusDays(rnd.nextInt(14)).minusMinutes(rnd.nextInt(600)));
        o.setTotal(group.stream().map(Ticket::getPrice).reduce(BigDecimal.ZERO, BigDecimal::add));
        orders.save(o);
        for (Ticket t : group) { t.setStatus(Ticket.Status.SOLD); t.setOrder(o); }
        i += n;
      }
    }
  }
}
```

backend/src/main/java/com/venuevista/service/EventService.java

```
package com.venuevista.service;
import com.venuevista.model.*;
import com.venuevista.repo.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDateTime;
import java.util.*;
@Service @RequiredArgsConstructor @Transactional
public class EventService {
  private final EventRepository events;
  private final VenueRepository venues;
  private final TicketRepository tickets;

  public record EventReq(String title, String category, String description, LocalDateTime startTime,
                         Long venueId, BigDecimal basePrice, String imageUrl) {}
  public record EventDto(Long id, String title, String category, String description, LocalDateTime startTime,
                         Long venueId, String venueName, String city, BigDecimal basePrice, String imageUrl,
                         String status, long totalSeats, long soldSeats) {}
  public record SeatDto(Long id, String row, int number, String tier, BigDecimal price, String status) {}

  private EventDto toDto(Event e, long[] c) {
    Venue v = e.getVenue();
    return new EventDto(e.getId(), e.getTitle(), e.getCategory(), e.getDescription(), e.getStartTime(), v.getId(),
      v.getName(), v.getCity(), e.getBasePrice(), e.getImageUrl(), e.getStatus().name(), c[0], c[1]);
  }

  private Map<Long, long[]> counts() {
    Map<Long, long[]> m = new HashMap<>();
    for (Object[] r : tickets.countsByEvent()) {
      long[] c = m.computeIfAbsent((Long) r[0], k -> new long[2]);
      long n = (Long) r[2];
      c[0] += n;
      if (r[1] == Ticket.Status.SOLD) c[1] += n;
    }
    return m;
  }

  @Transactional(readOnly = true)
  public List<EventDto> list(String q, String category) {
    Map<Long, long[]> counts = counts();
    String needle = q == null ? "" : q.trim().toLowerCase();
    return events.findAllWithVenue().stream()
      .filter(e -> needle.isEmpty() || e.getTitle().toLowerCase().contains(needle)
        || e.getVenue().getName().toLowerCase().contains(needle) || e.getVenue().getCity().toLowerCase().
contains(needle))
      .filter(e -> category == null || category.isBlank() || category.equalsIgnoreCase(e.getCategory()))
      .map(e -> toDto(e, counts.getOrDefault(e.getId(), new long[2]))).toList();
  }

  @Transactional(readOnly = true)
  public EventDto get(Long id) {
    Event e = events.findById(id).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
 "Event not found"));
    long total = 0, sold = 0;
    for (Ticket t : tickets.findByEventIdOrderByRowLabelAscSeatNumberAsc(id)) { total++; if (t.getStatus() == Ticket.
Status.SOLD) sold++; }
    return toDto(e, new long[]{total, sold});
  }

  @Transactional(readOnly = true)
  public List<SeatDto> seats(Long eventId) {
    return tickets.findByEventIdOrderByRowLabelAscSeatNumberAsc(eventId).stream()
      .map(t -> new SeatDto(t.getId(), t.getRowLabel(), t.getSeatNumber(), t.getTier().name(), t.getPrice(), t.
getStatus().name())).toList();
  }

  /** Creates the event and generates its seating inventory from the venue layout, priced by tier. */
  public EventDto create(EventReq r) {
    if (r.title() == null || r.title().isBlank() || r.startTime() == null || r.venueId() == null || r.
basePrice() == null)
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Title, date, venue and base price are required");
    Venue v = venues.findById(r.venueId()).orElseThrow(() -> new ResponseStatusException(HttpStatus.BAD_REQUEST,
 "Venue not found"));
    Event e = new Event();
    e.setTitle(r.title().trim()); e.setCategory(r.category()); e.setDescription(r.description());
    e.setStartTime(r.startTime()); e.setVenue(v); e.setBasePrice(r.basePrice()); e.setImageUrl(r.imageUrl());
    events.save(e);
    createSeats(e, v);
```

```
    return get(e.getId());
  }

  public void createSeats(Event e, Venue v) {
    int rows = v.getRowsCount();
    List<Ticket> list = new ArrayList<>();
    for (int i = 0; i < rows; i++) {
      Ticket.Tier tier = i < Math.ceil(rows * 0.2) ? Ticket.Tier.VIP : i < Math.ceil(rows * 0.6) ? Ticket.Tier.
PREMIUM : Ticket.Tier.STANDARD;
      double mult = tier == Ticket.Tier.VIP ? 2.0 : tier == Ticket.Tier.PREMIUM ? 1.5 : 1.0;
      BigDecimal price = e.getBasePrice().multiply(BigDecimal.valueOf(mult)).setScale(2, RoundingMode.HALF_UP);
      String label = String.valueOf((char) ('A' + i));
      for (int s = 1; s <= v.getSeatsPerRow(); s++) {
        Ticket t = new Ticket();
        t.setEvent(e); t.setRowLabel(label); t.setSeatNumber(s); t.setTier(tier); t.setPrice(price);
        list.add(t);
      }
    }
    tickets.saveAll(list);
  }

  public void cancel(Long id) {
    Event e = events.findById(id).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
 "Event not found"));
    e.setStatus(Event.Status.CANCELLED);
  }
}
```

backend/src/main/java/com/venuevista/service/OrderService.java

```
package com.venuevista.service;
import com.venuevista.model.*;
import com.venuevista.repo.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.*;
@Service @RequiredArgsConstructor @Transactional
public class OrderService {
  private final OrderRepository orders;
  private final TicketRepository tickets;
  private final EventRepository events;
  private final UserRepository users;

  public record OrderReq(Long eventId, List<Long> seatIds) {}
  public record SeatLine(String label, String tier, BigDecimal price) {}
  public record OrderDto(Long id, String code, Long eventId, String eventTitle, LocalDateTime startTime,
 String venueName,
                         String city, BigDecimal total, LocalDateTime createdAt, List<SeatLine> seats) {}

  public OrderDto place(String email, OrderReq r) {
    AppUser u = users.findByEmail(email).orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
    if (r.seatIds() == null || r.seatIds().isEmpty() || r.seatIds().size() > 8)
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Choose between 1 and 8 seats");
    Event ev = events.findById(r.eventId()).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
 "Event not found"));
    if (ev.getStatus() != Event.Status.ACTIVE) throw new ResponseStatusException(HttpStatus.CONFLICT,
 "This event has been cancelled");
    if (ev.getStartTime().isBefore(LocalDateTime.now())) throw new ResponseStatusException(HttpStatus.CONFLICT,
 "This event has already started");

    List<Ticket> ts = tickets.lockAll(new ArrayList<>(new LinkedHashSet<>(r.seatIds())));  //
 row locks prevent double-selling
    if (ts.size() != new HashSet<>(r.seatIds()).size() || ts.stream().anyMatch(t -> !t.getEvent().getId().equals(ev.
getId())))
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Some seats don't belong to this event");
    if (ts.stream().anyMatch(t -> t.getStatus() != Ticket.Status.AVAILABLE))
      throw new ResponseStatusException(HttpStatus.CONFLICT, "Someone just booked one of those seats.
 Pick different seats.");

    BookingOrder o = new BookingOrder();
    o.setUser(u); o.setEvent(ev); o.setCode("VV-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase());
    o.setTotal(ts.stream().map(Ticket::getPrice).reduce(BigDecimal.ZERO, BigDecimal::add));
    orders.save(o);
    for (Ticket t : ts) { t.setStatus(Ticket.Status.SOLD); t.setOrder(o); }
    o.setTickets(ts);
    return toDto(o);
  }

  @Transactional(readOnly = true)
  public List<OrderDto> mine(String email) {
    AppUser u = users.findByEmail(email).orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
    return orders.findByUserIdOrderByCreatedAtDesc(u.getId()).stream().map(this::toDto).toList();
  }

  OrderDto toDto(BookingOrder o) {
    Event e = o.getEvent();
    List<SeatLine> seats = o.getTickets().stream()
      .sorted(Comparator.comparing(Ticket::getRowLabel).thenComparingInt(Ticket::getSeatNumber))
      .map(t -> new SeatLine(t.label(), t.getTier().name(), t.getPrice())).toList();
    return new OrderDto(o.getId(), o.getCode(), e.getId(), e.getTitle(), e.getStartTime(), e.getVenue().getName(),
      e.getVenue().getCity(), o.getTotal(), o.getCreatedAt(), seats);
  }
}
```

backend/src/main/java/com/venuevista/VenueVistaApplication.java

```
package com.venuevista;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class VenueVistaApplication {
  public static void main(String[] args) { SpringApplication.run(VenueVistaApplication.class, args); }
}
```

backend/src/main/java/com/venuevista/web/AnalyticsController.java

```
package com.venuevista.web;
import com.venuevista.service.AnalyticsService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
@RestController @RequestMapping("/api/analytics") @RequiredArgsConstructor
public class AnalyticsController {
  private final AnalyticsService svc;
  @GetMapping("/summary") public AnalyticsService.Summary summary() { return svc.summary(); }
}
```

backend/src/main/java/com/venuevista/web/AuthController.java

```
package com.venuevista.web;
import com.venuevista.model.AppUser;
import com.venuevista.repo.UserRepository;
import com.venuevista.security.JwtUtil;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
@RestController @RequestMapping("/api/auth") @RequiredArgsConstructor
public class AuthController {
  private final UserRepository users;
  private final PasswordEncoder encoder;
  private final JwtUtil jwt;
  public record RegisterReq(@NotBlank String name, @Email @NotBlank String email, @Size(min = 8) String password,
 String role) {}
  public record LoginReq(@NotBlank String email, @NotBlank String password) {}
  public record AuthRes(String token, String name, String email, String role) {}

  @PostMapping("/register")
  public AuthRes register(@Valid @RequestBody RegisterReq r) {
    String email = r.email().trim().toLowerCase();
    if (users.existsByEmail(email)) throw new ResponseStatusException(HttpStatus.CONFLICT,
 "An account with this email already exists");
    AppUser u = new AppUser();
    u.setName(r.name().trim()); u.setEmail(email); u.setPasswordHash(encoder.encode(r.password()));
    u.setRole("ORGANIZER".equalsIgnoreCase(r.role()) ? AppUser.Role.ORGANIZER : AppUser.Role.CUSTOMER);  //
 ADMIN can't self-register
    users.save(u);
    return res(u);
  }

  @PostMapping("/login")
  public AuthRes login(@Valid @RequestBody LoginReq r) {
    AppUser u = users.findByEmail(r.email().trim().toLowerCase()).orElse(null);
    if (u == null || !encoder.matches(r.password(), u.getPasswordHash()))
      throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Email or password is incorrect");
    return res(u);
  }

  private AuthRes res(AppUser u) { return new AuthRes(jwt.generate(u.getEmail(), u.getRole().name()), u.getName(), u.
getEmail(), u.getRole().name()); }
}
```

backend/src/main/java/com/venuevista/web/EventController.java

```
package com.venuevista.web;
import com.venuevista.service.EventService;
import com.venuevista.service.EventService.*;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController @RequestMapping("/api/events") @RequiredArgsConstructor
public class EventController {
  private final EventService svc;
  @GetMapping public List<EventDto> list(@RequestParam(required = false) String q,
 @RequestParam(required = false) String category) { return svc.list(q, category); }
  @GetMapping("/{id}") public EventDto get(@PathVariable Long id) { return svc.get(id); }
  @GetMapping("/{id}/seats") public List<SeatDto> seats(@PathVariable Long id) { return svc.seats(id); }
  @PostMapping public EventDto create(@RequestBody EventReq r) { return svc.create(r); }
  @DeleteMapping("/{id}") public void cancel(@PathVariable Long id) { svc.cancel(id); }
}
```

backend/src/main/java/com/venuevista/web/OrderController.java

```
package com.venuevista.web;
import com.venuevista.service.OrderService;
import com.venuevista.service.OrderService.*;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController @RequestMapping("/api/orders") @RequiredArgsConstructor
public class OrderController {
  private final OrderService svc;
  @PostMapping public OrderDto place(Authentication a, @RequestBody OrderReq r) { return svc.place(a.getName(), r); }
  @GetMapping("/mine") public List<OrderDto> mine(Authentication a) { return svc.mine(a.getName()); }
}
```

backend/src/main/java/com/venuevista/web/VenueController.java

```
package com.venuevista.web;
import com.venuevista.model.Venue;
import com.venuevista.repo.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
import java.util.List;
@RestController @RequestMapping("/api/venues") @RequiredArgsConstructor
public class VenueController {
  private final VenueRepository venues;
  private final EventRepository events;
  @GetMapping public List<Venue> list() { return venues.findAll(); }
  @PostMapping
  public Venue create(@RequestBody Venue v) {
    if (v.getName() == null || v.getName().isBlank()) throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
 "Venue name is required");
    if (v.getRowsCount() < 1 || v.getRowsCount() > 26 || v.getSeatsPerRow() < 1 || v.getSeatsPerRow() > 40)
      throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Use 1-26 rows and 1-40 seats per row");
    v.setId(null);
    return venues.save(v);
  }
  @DeleteMapping("/{id}")
  public void delete(@PathVariable Long id) {
    if (events.countByVenueId(id) > 0) throw new ResponseStatusException(HttpStatus.CONFLICT,
 "This venue has events and can't be removed");
    venues.deleteById(id);
  }
}
```

backend/src/main/resources/application-local.yml

```
spring:
  datasource:
    url: jdbc:h2:mem:venuevista;MODE=Oracle;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
    username: sa
    password:
    driver-class-name: org.h2.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
    open-in-view: false
```

backend/src/main/resources/application.yml

```
spring:
  config:
    import: "optional:file:../.env[.properties],optional:file:.env[.properties]"
  datasource:
    url: ${DB_URL:jdbc:oracle:thin:@//localhost:1521/XEPDB1}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    driver-class-name: oracle.jdbc.OracleDriver
  jpa:
    hibernate:
      ddl-auto: update        # creates/updates tables automatically on first run
    open-in-view: false
server:
  port: 8081
  error:
    include-message: always
app:
  cors-origin: ${CORS_ORIGIN:http://localhost:4200}
  jwt:
    secret: ${JWT_SECRET}
    expiration-ms: ${JWT_EXPIRATION_MS:86400000}
```

database/01_create_user.sql

```
-- Run as SYSTEM / SYS against your pluggable DB (e.g. XEPDB1). Tables are created by Hibernate on first start.
ACCEPT venuevista_password CHAR PROMPT 'Enter password for VENUEVISTA: ' HIDE
CREATE USER venuevista IDENTIFIED BY "&&venuevista_password" DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;
GRANT CONNECT, RESOURCE, CREATE VIEW TO venuevista;
UNDEFINE venuevista_password
```

frontend/angular.json

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1, "newProjectRoot": "projects",
  "projects": { "venuevista": {
    "projectType": "application", "root": "", "sourceRoot": "src", "prefix": "vv",
    "architect": {
      "build": { "builder": "@angular-devkit/build-angular:application",
        "options": { "outputPath": "dist/venuevista", "index": "src/index.html", "browser": "src/main.ts",
          "polyfills": ["zone.js"], "tsConfig": "tsconfig.app.json", "assets": [], "styles": ["src/styles.scss"],
 "scripts": [] },
        "configurations": {
          "production": { "budgets": [{ "type": "initial", "maximumWarning": "1mb", "maximumError": "2mb" }],
 "outputHashing": "all" },
          "development": { "optimization": false, "extractLicenses": false, "sourceMap": true } },
        "defaultConfiguration": "production" },
      "serve": { "builder": "@angular-devkit/build-angular:dev-server",
        "options": { "proxyConfig": "proxy.conf.json" },
        "configurations": { "production": { "buildTarget": "venuevista:build:production" }, "development":
 { "buildTarget": "venuevista:build:development" } },
        "defaultConfiguration": "development" }
    } } }
}
```

frontend/package.json

```
{
  "name": "venuevista-web",
  "version": "1.0.0",
  "scripts": { "start": "ng serve --open", "build": "ng build" },
  "dependencies": {
    "@angular/animations": "^17.3.0", "@angular/common": "^17.3.0", "@angular/compiler": "^17.3.0",
    "@angular/core": "^17.3.0", "@angular/forms": "^17.3.0", "@angular/platform-browser": "^17.3.0",
    "@angular/platform-browser-dynamic": "^17.3.0", "@angular/router": "^17.3.0",
    "rxjs": "~7.8.0", "tslib": "^2.6.0", "zone.js": "~0.14.3"
  },
  "devDependencies": {
    "@angular-devkit/build-angular": "^17.3.0", "@angular/cli": "^17.3.0", "@angular/compiler-cli": "^17.3.0",
    "typescript": "~5.4.2"
  }
}
```

frontend/proxy.conf.json

```
{
  "/api": {
    "target": "http://localhost:8081",
    "secure": false,
    "changeOrigin": true
  }
}
```

frontend/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
@Component({ selector: 'app-root', standalone: true, imports: [RouterOutlet], template: '<router-outlet />' })
export class AppComponent {}
```

frontend/src/app/app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './core/auth.interceptor';
export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withComponentInputBinding()),
 provideHttpClient(withInterceptors([authInterceptor]))]
};
```

frontend/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { authGuard, staffGuard } from './core/guards';
export const routes: Routes = [
  { path: 'login', loadComponent: () => import('./pages/login.component').then(m => m.LoginComponent) },
  { path: 'register', loadComponent: () => import('./pages/register.component').then(m => m.RegisterComponent) },
  { path: '', loadComponent: () => import('./pages/shell.component').then(m => m.ShellComponent), children: [
    { path: '', pathMatch: 'full', redirectTo: 'events' },
    { path: 'events', loadComponent: () => import('./pages/events.component').then(m => m.EventsComponent) },
    { path: 'events/new', canActivate: [staffGuard], loadComponent: () => import('./pages/event-form.component').
then(m => m.EventFormComponent) },
    { path: 'events/:id', loadComponent: () => import('./pages/event-detail.component').then(m => m.
EventDetailComponent) },
    { path: 'tickets', canActivate: [authGuard], loadComponent: () => import('./pages/my-tickets.component').
then(m => m.MyTicketsComponent) },
    { path: 'dashboard', canActivate: [staffGuard], loadComponent: () => import('./pages/dashboard.component').
then(m => m.DashboardComponent) },
    { path: 'venues', canActivate: [staffGuard], loadComponent: () => import('./pages/venues.component').then(m => m.
VenuesComponent) },
  ] },
  { path: '**', redirectTo: '' }
];
```

frontend/src/app/core/api.service.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
export const API = '/api';

export interface EventItem { id: number; title: string; category: string; description: string; startTime: string;
 venueId: number;
  venueName: string; city: string; basePrice: number; imageUrl?: string; status: string; totalSeats: number;
 soldSeats: number; }
export type Tier = 'STANDARD' | 'PREMIUM' | 'VIP';
export interface Seat { id: number; row: string; number: number; tier: Tier; price: number; status:
 'AVAILABLE' | 'SOLD'; }
export interface Venue { id: number; name: string; city: string; address: string; rowsCount: number; seatsPerRow:
 number; capacity: number; }
export interface SeatLine { label: string; tier: Tier; price: number; }
export interface Order { id: number; code: string; eventId: number; eventTitle: string; startTime: string; venueName:
 string; city: string;
  total: number; createdAt: string; seats: SeatLine[]; }
export interface Summary {
  kpis: { revenue: number; ticketsSold: number; totalSeats: number; occupancy: number; events: number; avgOrder:
 number };
  topEvents: { title: string; revenue: number; sold: number; capacity: number }[];
  daily: { date: string; revenue: number; orders: number }[];
  soldByTier: Record<Tier, number>;
}

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  events(q = '', category = '') {
    let p = new HttpParams(); if (q) p = p.set('q', q); if (category) p = p.set('category', category);
    return this.http.get<EventItem[]>(`${API}/events`, { params: p });
  }
  event(id: number) { return this.http.get<EventItem>(`${API}/events/${id}`); }
  seats(id: number) { return this.http.get<Seat[]>(`${API}/events/${id}/seats`); }
  createEvent(b: unknown) { return this.http.post<EventItem>(`${API}/events`, b); }
  cancelEvent(id: number) { return this.http.delete<void>(`${API}/events/${id}`); }
  venues() { return this.http.get<Venue[]>(`${API}/venues`); }
  createVenue(b: unknown) { return this.http.post<Venue>(`${API}/venues`, b); }
  deleteVenue(id: number) { return this.http.delete<void>(`${API}/venues/${id}`); }
  placeOrder(eventId: number, seatIds: number[]) { return this.http.post<Order>(`${API}/orders`, { eventId,
 seatIds }); }
  myOrders() { return this.http.get<Order[]>(`${API}/orders/mine`); }
  summary() { return this.http.get<Summary>(`${API}/analytics/summary`); }
}
export function errorText(e: any, fallback = 'Something went wrong. Please try again.'): string {
  return e?.error?.message || (e?.status === 0 ? 'Can’t reach the server. Check that the API is running.' : fallback);
}
```

frontend/src/app/core/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from './auth.service';
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.user()?.token;
  const r = token && !req.url.includes('/auth/') ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) :
 req;
  return next(r).pipe(catchError(e => {
    if (e.status === 401 && token && !req.url.includes('/auth/')) auth.logout();   // expired session
    return throwError(() => e);
  }));
};
```

frontend/src/app/core/auth.service.ts

```
import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { tap } from 'rxjs';
import { API } from './api.service';
export interface AuthUser { token: string; name: string; email: string; role: 'ADMIN' | 'ORGANIZER' | 'CUSTOMER'; }
@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient); private router = inject(Router);
  user = signal<AuthUser | null>(JSON.parse(localStorage.getItem('vv_user') || 'null'));
  isStaff = computed(() => ['ADMIN', 'ORGANIZER'].includes(this.user()?.role ?? ''));
  login(b: { email: string; password: string }) { return this.http.post<AuthUser>(`${API}/auth/login`, b).
pipe(tap(u => this.store(u))); }
  register(b: { name: string; email: string; password: string; role: string }) { return this.http.
post<AuthUser>(`${API}/auth/register`, b).pipe(tap(u => this.store(u))); }
  logout() { localStorage.removeItem('vv_user'); this.user.set(null); this.router.navigate(['/login']); }
  private store(u: AuthUser) { localStorage.setItem('vv_user', JSON.stringify(u)); this.user.set(u); }
}
```

frontend/src/app/core/guards.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';
export const authGuard: CanActivateFn = () => { const a = inject(AuthService); return a.user() ? true :
 inject(Router).createUrlTree(['/login']); };
export const staffGuard: CanActivateFn = () => { const a = inject(AuthService); return a.user() ? (a.isStaff() ?
 true : inject(Router).createUrlTree(['/events'])) : inject(Router).createUrlTree(['/login']); };
```

frontend/src/app/core/util.ts

```
export const CATEGORIES = ['Concert', 'Comedy', 'Theatre', 'Festival', 'Sports', 'Conference'];
const HUES: Record<string, string> = { Concert: '#3B3A8F', Comedy: '#B8862B', Theatre: '#8E1230', Festival: '#1D6E8C',
 Sports: '#1F7A4D', Conference: '#4B4F6B' };
export const catColor = (c: string) => HUES[c] ?? '#3A3768';
```

frontend/src/app/pages/dashboard.component.ts

```
import { Component, OnInit, computed, inject, signal } from '@angular/core';
import { CurrencyPipe, DatePipe, DecimalPipe } from '@angular/common';
import { ApiService, Summary, errorText } from '../core/api.service';
@Component({
  selector: 'vv-dashboard', standalone: true, imports: [CurrencyPipe, DatePipe, DecimalPipe],
  template: `
  <div class="page">
    <div class="page-head"><div><h1>Sales analytics</h1><p class="muted">Revenue,
 seat fill and tier mix across all events.</p></div></div>
    @if (error()) { <div class="alert">{{ error() }}</div> }
    @if (data(); as d) {
      <div class="kpis">
        <div class="kpi main"><span>Total revenue</span><b>{{ d.kpis.revenue | currency:'INR':'symbol':'1.0-0' }}</
b><small>from {{ d.kpis.ticketsSold | number }} tickets</small></div>
        <div class="kpi"><span>Seat fill</span><b>{{ d.kpis.occupancy }}%</b><small>{{ d.kpis.
ticketsSold | number }} of {{ d.kpis.totalSeats | number }} seats</small></div>
        <div class="kpi"><span>Average order</span><b>{{ d.kpis.avgOrder | currency:'INR':'symbol':'1.0-0' }}</
b><small>across all bookings</small></div>
        <div class="kpi"><span>Events</span><b>{{ d.kpis.events }}</b><small>listed</small></div>
      </div>
      <div class="cols">
        <section class="panel">
          <h2>Revenue, last 14 days</h2>
          <svg viewBox="0 0 600 190" role="img" aria-label="Daily revenue line chart" preserveAspectRatio="none">
            <defs><linearGradient id="g" x1="0" y1="0" x2="0" y2="1"><stop offset="0" stop-color="#C4122F" stop-opacit
y=".25"/><stop offset="1" stop-color="#C4122F" stop-opacity="0"/></linearGradient></defs>
            <path [attr.d]="area()" fill="url(#g)"/><path [attr.
d]="line()" fill="none" stroke="#C4122F" stroke-width="2.5" vector-effect="non-scaling-stroke"/>
          </svg>
          <div class="axis"><span>{{ d.daily[0].date | date:'d MMM' }}</span><span>Peak {{ peak() | currency:'INR':
'symbol':'1.0-0' }}</span><span>{{ d.daily[d.daily.length-1].date | date:'d MMM' }}</span></div>
        </section>
        <section class="panel">
          <h2>Tickets by tier</h2>
          <div class="donut-wrap"><div class="donut" [style.
background]="donut()" role="img" aria-label="Share of tickets sold by tier"><span>{{ d.kpis.ticketsSold | number }}</
span></div>
            <ul>@for (t of tierList(); track t.k) { <li><i class="tier-dot" [attr.data-tier]="t.k"></i>{{ t.k.
charAt(0) + t.k.slice(1).toLowerCase() }} <b>{{ t.n | number }}</b></li> }</ul></div>
        </section>
      </div>
      <section class="panel">
        <h2>Top events by revenue</h2>
        @if (!d.topEvents.length) { <p class="muted">No sales yet.</p> }
        @for (e of d.topEvents; track e.title) {
          <div class="ev"><div class="top"><span>{{ e.title }}</span><b>{{ e.revenue | currency:'INR':'symbol':'1.
0-0' }}</b></div>
            <div class="bar"><i [style.width.%]="e.revenue * 100 / d.topEvents[0].revenue"></i></div>
            <small class="muted">{{ e.sold }} / {{ e.capacity }} seats sold ({{ e.capacity ? (e.sold * 100 / e.
capacity | number:'1.0-0') : 0 }}%)</small></div>
        }
      </section>
    } @else if (!error()) { <div class="skeleton" style="height:320px"></div> }
  </div>`,
  styles: [`
    .kpis { display:grid; grid-template-columns:1.4fr 1fr 1fr 1fr; gap:16px; margin-bottom:22px; }
    .kpi { background:#fff; border:1px solid var(--line); border-radius:var(--r-md); padding:18px; display:flex;
 flex-direction:column; gap:2px; }
    .kpi span { color:var(--muted); font-weight:500; } .kpi b { font:800 1.9rem var(--head); } .kpi small { color:
var(--muted); }
    .kpi.main { background:var(--ink); color:#fff; border:0; } .kpi.main span,.kpi.main small { color:#B9BBDD; } .kpi.
main b { font-size:2.4rem; }
    .cols { display:grid; grid-template-columns:1.7fr 1fr; gap:16px; margin-bottom:16px; } h2 { margin-bottom:14px; }
    svg { width:100%; height:190px; display:block; } .axis { display:flex; justify-content:space-between; font-size:
12.5px; color:var(--muted); margin-top:6px; }
    .donut-wrap { display:flex; align-items:center; gap:22px; flex-wrap:wrap; }
    .donut { width:140px; height:140px; border-radius:50%; display:grid; place-items:center; position:relative; }
    .donut::after { content:''; position:absolute; inset:24px; background:#fff; border-radius:50%; } .
donut span { position:relative; z-index:1; font:800 1.3rem var(--head); }
    ul { list-style:none; margin:0; padding:0; display:grid; gap:8px; } li b { margin-left:6px; }
    .ev { margin-bottom:16px; } .top { display:flex; justify-content:space-between; margin-bottom:6px; } .
bar { height:10px; background:#E4E7F3; border-radius:6px; overflow:hidden; margin-bottom:4px; }
    .bar i { display:block; height:100%; background:var(--ink-3); border-radius:6px; }
    @media (max-width:900px) { .kpis { grid-template-columns:1fr 1fr; } .cols { grid-template-columns:1fr; } }`]
})
export class DashboardComponent implements OnInit {
  private api = inject(ApiService);
  data = signal<Summary | null>(null); error = signal('');
  private pts = computed(() => {
    const d = this.data()?.daily ?? []; const max = Math.max(...d.map(x => x.revenue), 1);
    return d.map((x, i) => [i * (600 / Math.max(d.length - 1, 1)), 180 - (x.revenue / max) * 160]);
  });
  line = computed(() => this.pts().map((p, i) => `${i ? 'L' : 'M'}${p[0].toFixed(1)} ${p[1].toFixed(1)}`).join(' '));
```

```
  area = computed(() => this.line() + ' L600 190 L0 190 Z');
  peak = computed(() => Math.max(...(this.data()?.daily ?? []).map(x => x.revenue), 0));
  tierList = computed(() => Object.entries(this.data()?.soldByTier ?? {}).map(([k, n]) => ({ k, n })));
  donut = computed(() => {
    const t = this.data()?.soldByTier; if (!t) return '';
    const sum = (t.STANDARD + t.PREMIUM + t.VIP) || 1; const a = t.STANDARD * 100 / sum; const b = a + t.
PREMIUM * 100 / sum;
    return `conic-gradient(#5F6FD8 0 ${a}%, #B8862B ${a}% ${b}%, #C4122F ${b}% 100%)`;
  });
  ngOnInit() { this.api.summary().subscribe({ next: d => this.data.set(d), error: e => this.error.
set(errorText(e)) }); }
}
```

frontend/src/app/pages/event-detail.component.ts

```
import { Component, Input, OnInit, computed, inject, signal } from '@angular/core';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { Router, RouterLink } from '@angular/router';
import { ApiService, EventItem, Seat, errorText } from '../core/api.service';
import { AuthService } from '../core/auth.service';
import { catColor } from '../core/util';
@Component({
  selector: 'vv-event-detail', standalone: true, imports: [DatePipe, CurrencyPipe, RouterLink],
  template: `
  <div class="page">
    <a routerLink="/events" class="muted back">Back to events</a>
    @if (event(); as e) {
      <header [style.background]="color(e.category)">
        <div><span class="chip">{{ e.category }}</span><h1>{{ e.title }}</h1>
          <p>{{ e.startTime | date:'EEEE, d MMMM y · h:mm a' }}</p><p>{{ e.venueName }}, {{ e.city }}</p></div>
        @if (auth.isStaff() && e.
status === 'ACTIVE') { <button class="btn danger sm" (click)="cancel(e)">Cancel event</button> }
      </header>
      <p class="desc">{{ e.description }}</p>
      @if (e.status === 'CANCELLED') { <div class="alert">This event has been cancelled. Seat selling is closed.</
div> }
      <div class="layout">
        <section class="panel chart">
          <div class="legend">
            @for (t of tiers; track t.k) { <span><i class="tier-dot" [attr.data-tier]="t.k"></i>{{ t.k.charAt(0) + t.
k.slice(1).toLowerCase() }} {{ t.price | currency:'INR':'symbol':'1.0-0' }}</span> }
            <span><i class="sold-key"></i>Taken</span><span><i class="sel-key"></i>Your pick</span>
          </div>
          <div class="scroll">
            <div class="stage">STAGE</div>
            @for (r of rows(); track r.label) {
              <div class="srow"><span class="rl">{{ r.label }}</span>
                @for (s of r.seats; track s.id) {
                  <button class="seat" [attr.data-tier]="s.tier" [class.sold]="s.status==='SOLD'" [class.
sel]="isSel(s)"
                    [class.aisle]="s.number === mid()" [disabled]="s.status==='SOLD' || e.status!
=='ACTIVE'" (click)="toggle(s)"
                    [attr.aria-pressed]="isSel(s)" [attr.aria-label]="'Row ' + s.row + ' seat ' + s.number + ', ' + s.
tier.toLowerCase() + ', ' + s.price + ' rupees' + (s.status==='SOLD' ? ', taken' : '')"></button>
                }
                <span class="rl">{{ r.label }}</span>
              </div>
            }
          </div>
        </section>
        <aside class="panel cart">
          <h2>Your seats</h2>
          @if (!selected().length) { <p class="muted">Select up to 8 seats on the map. Prices vary by tier.</p> }
          @else {
            <ul>@for (s of selected(); track s.id) { <li><span><i class="tier-dot" [attr.data-tier]="s.tier"></
i>Row {{ s.row }}, seat {{ s.number }}</span><b>{{ s.price | currency:'INR':'symbol':'1.0-0' }}</b></li> }</ul>
            <div class="total"><span>Total</span><b>{{ total() | currency:'INR':'symbol':'1.0-0' }}</b></div>
          }
          @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
          <button class="btn block" [disabled]="!selected().length || busy() || e.status!
=='ACTIVE'" (click)="checkout(e)">
            {{ busy() ? 'Booking…' : auth.user() ? 'Book ' + selected().length + ' seat' + (selected().length===1?'':
's') : 'Sign in to book' }}</button>
          <p class="hint">{{ e.totalSeats - e.soldSeats }} of {{ e.totalSeats }} seats still available.</p>
        </aside>
      </div>
    } @else if (error()) { <div class="alert">{{ error() }}</div> } @else { <div class="skeleton" style="height:
240px"></div> }
  </div>`,
  styles: [`
    .back { display:inline-block; margin-bottom:12px; text-decoration:none; }
    header { color:#fff; border-radius:var(--r-lg); padding:32px; display:flex; justify-content:space-between; gap:
16px; align-items:flex-start; }
    header h1 { margin:10px 0; } header p { opacity:.9; } header .chip { background:#ffffff2b; color:#fff; }
    header .btn.danger { color:#fff; box-shadow:inset 0 0 0 1.5px #fff; } .desc { margin:20px 0; max-width:70ch; }
    .layout { display:grid; grid-template-columns:1fr 330px; gap:22px; align-items:start; }
    .legend { display:flex; gap:18px; flex-wrap:wrap; font-size:13.5px; margin-bottom:18px; } .legend span { display:
flex; align-items:center; }
    .sold-key,.sel-key { width:12px; height:12px; border-radius:3px; display:inline-block; margin-right:6px; }
    .sold-key { background:#CFD2E3; } .sel-key { background:var(--ink); }
    .scroll { overflow-x:auto; padding-bottom:8px; display:flex; flex-direction:column; align-items:center; min-width:
0; }
    .stage { width:70%; min-width:260px; text-align:center; padding:8px; background:var(--ink); color:#fff;
 border-radius:4px 4px 50% 50% / 4px 4px 14px 14px; font:700 12.5px var(--head); letter-spacing:.2em; margin-bottom:
26px; }
    .srow { display:flex; gap:5px; align-items:center; margin-bottom:5px; } .rl { width:20px; text-align:center;
 font-size:12px; color:var(--muted); font-weight:600; }
    .seat { width:24px; height:24px; border:0; border-radius:5px 5px 9px 9px; cursor:pointer; padding:0; transition:
```

```
transform .1s; }
    .seat[data-tier=STANDARD] { background:var(--standard); } .seat[data-tier=PREMIUM] { background:var(--premium);
 } .seat[data-tier=VIP] { background:var(--vip); }
    .seat:hover:not(:disabled) { transform:scale(1.2); } .seat.aisle { margin-right:14px; }
    .seat.sold { background:#CFD2E3 !important; cursor:not-allowed; } .seat.sel { background:var(--ink) !important;
 box-shadow:0 0 0 2px #fff, 0 0 0 4px var(--ink); }
    .cart { position:sticky; top:20px; } .cart h2 { margin-bottom:12px; } .cart ul { list-style:none; padding:0;
 margin:0 0 12px; }
    .cart li { display:flex; justify-content:space-between; padding:8px 0; border-bottom:1px dashed var(--line); }
    .total { display:flex; justify-content:space-between; font:700 1.2rem var(--head); margin:12px 0 16px; } .cart .
btn { margin-top:6px; } .cart .hint { margin-top:10px; }
    @media (max-width:980px) { .layout { grid-template-columns:1fr; } .cart { position:static; } }`]
})
export class EventDetailComponent implements OnInit {
  @Input() id!: string;
  private api = inject(ApiService); private router = inject(Router); auth = inject(AuthService);
  event = signal<EventItem | null>(null); seats = signal<Seat[]>([]); selected = signal<Seat[]>([]);
  error = signal(''); busy = signal(false); color = catColor;
  total = computed(() => this.selected().reduce((a, s) => a + s.price, 0));
  rows = computed(() => {
    const m = new Map<string, Seat[]>(); this.seats().forEach(s => (m.get(s.row) ?? m.set(s.row, []).get(s.row)!).
push(s));
    return [...m].map(([label, seats]) => ({ label, seats }));
  });
  mid = computed(() => Math.floor((this.rows()[0]?.seats.length ?? 0) / 2));
  tiers = [{ k: 'STANDARD', price: 0 }, { k: 'PREMIUM', price: 0 }, { k: 'VIP', price: 0 }];
  ngOnInit() {
    this.api.event(+this.id).subscribe({ next: e => this.event.set(e), error: e => this.error.set(errorText(e,
 'Event not found.')) });
    this.loadSeats();
  }
  private loadSeats() {
    this.api.seats(+this.id).subscribe(s => {
      this.seats.set(s);
      this.tiers = this.tiers.map(t => ({ ...t, price: s.find(x => x.tier === t.k)?.price ?? 0 }));
      const open = new Set(s.filter(x => x.status === 'AVAILABLE').map(x => x.id));
      this.selected.update(sel => sel.filter(x => open.has(x.id)));
    });
  }
  isSel(s: Seat) { return this.selected().some(x => x.id === s.id); }
  toggle(s: Seat) {
    this.error.set('');
    if (this.isSel(s)) return this.selected.update(l => l.filter(x => x.id !== s.id));
    if (this.selected().length >= 8) return this.error.set('You can book up to 8 seats at a time.');
    this.selected.update(l => [...l, s]);
  }
  checkout(e: EventItem) {
    if (!this.auth.user()) { this.router.navigate(['/login']); return; }
    this.busy.set(true); this.error.set('');
    this.api.placeOrder(e.id, this.selected().map(s => s.id)).subscribe({
      next: () => this.router.navigate(['/tickets']),
      error: err => { this.error.set(errorText(err)); this.busy.set(false); this.loadSeats(); }   //
 refresh so taken seats grey out
    });
  }
  cancel(e: EventItem) {
    if (confirm(`Cancel “${e.title}”? Buyers will no longer be able to book seats.`))
      this.api.cancelEvent(e.id).subscribe(() => this.ngOnInit());
  }
}
```

frontend/src/app/pages/event-form.component.ts

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { ApiService, Venue, errorText } from '../core/api.service';
import { CATEGORIES } from '../core/util';
@Component({
  selector: 'vv-event-form', standalone: true, imports: [FormsModule, RouterLink, CurrencyPipe],
  template: `
  <div class="page narrow">
    <div class="page-head"><div><h1>Create event</
h1><p class="muted">Seats are generated from the venue’s seating chart and priced by tier.</p></div></div>
    @if (!venues().length) { <div class="alert">You need a venue first. <a routerLink="/venues">Add a venue</a></
div> }
    <form class="panel" (ngSubmit)="save()" #f="ngForm">
      @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
      <div class="field"><label for="t">Event title</label><input id="t" name="title" [(ngModel)]="m.
title" required></div>
      <div class="two">
        <div class="field"><label for="c">Category</label><select id="c" name="category" [(ngModel)]="m.
category">@for (c of cats; track c) { <option [value]="c">{{ c }}</option> }</select></div>
        <div class="field"><label for="d">Date and start time</
label><input id="d" name="start" type="datetime-local" [(ngModel)]="m.startTime" required></div>
      </div>
      <div class="two">
        <div class="field"><label for="v">Venue</label><select id="v" name="venue" [(ngModel)]="m.venueId" required>
          <option [ngValue]="null" disabled>Choose a venue</option>
          @for (v of venues(); track v.id) { <option [ngValue]="v.id">{{ v.name }} ({{ v.capacity }} seats)</
option> }</select></div>
        <div class="field"><label for="p">Standard ticket price (■)</
label><input id="p" name="price" type="number" min="0" [(ngModel)]="m.basePrice" required>
          <span class="hint">Premium is 1.5× and VIP is 2× this price.</span></div>
      </div>
      <div class="field"><label for="ds">Description</label><textarea id="ds" name="desc" rows="4" [(ngModel)]="m.
description"></textarea></div>
      @if (m.basePrice) { <p class="hint prices">Standard {{ m.basePrice | currency:'INR':'symbol':'1.
0-0' }} · Premium {{ m.basePrice*1.5 | currency:'INR':'symbol':'1.0-0' }} · VIP {{ m.basePrice*2 | currency:'INR':
'symbol':'1.0-0' }}</p> }
      <button class="btn" [disabled]="f.invalid || busy()">{{ busy() ? 'Publishing…' : 'Publish event' }}</button>
    </form>
  </div>`,
  styles: [`.narrow { max-width:760px; } .two { display:grid; grid-template-columns:1fr 1fr; gap:16px; } .
prices { margin-bottom:16px; } @media (max-width:640px){ .two{grid-template-columns:1fr;} }`]
})
export class EventFormComponent implements OnInit {
  private api = inject(ApiService); private router = inject(Router);
  venues = signal<Venue[]>([]); error = signal(''); busy = signal(false); cats = CATEGORIES;
  m: any = { title: '', category: 'Concert', startTime: '', venueId: null, basePrice: null, description: '' };
  ngOnInit() { this.api.venues().subscribe(v => this.venues.set(v)); }
  save() {
    this.busy.set(true); this.error.set('');
    this.api.createEvent(this.m).subscribe({
      next: e => this.router.navigate(['/events', e.id]),
      error: e => { this.error.set(errorText(e)); this.busy.set(false); }
    });
  }
}
```

frontend/src/app/pages/events.component.ts

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterLink } from '@angular/router';
import { ApiService, EventItem, errorText } from '../core/api.service';
import { CATEGORIES, catColor } from '../core/util';
@Component({
  selector: 'vv-events', standalone: true, imports: [FormsModule, RouterLink, DatePipe, CurrencyPipe],
  template: `
  <div class="page">
    <div class="page-head"><div><h1>What’s on</h1><p class="muted">Pick an event, then pick your exact seats.</p></
div>
      <div class="row"><input class="search" type="search" placeholder="Search by event,
 venue or city" aria-label="Search events" [(ngModel)]="q" (ngModelChange)="load()"></div></div>
    <div class="cats" role="group" aria-label="Filter by category">
      <button [class.on]="!cat" (click)="setCat('')">All</button>
      @for (c of cats; track c) { <button [class.on]="cat===c" (click)="setCat(c)">{{ c }}</button> }
    </div>
    @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
    @if (loading()) { <div class="grid">@for (i of [1,2,3]; track i) { <div class="skeleton" style="height:300px"></
div> }</div> }
    @else if (!events().length) { <div class="empty"><h3>No events match that search</
h3><p>Try a different keyword or clear the category filter.</p></div> }
    @else {
      <div class="grid">
        @for (e of events(); track e.id) {
          <a class="card" [routerLink]="['/events', e.id]">
            <div class="poster" [style.background]="color(e.category)">
              <div class="date"><b>{{ e.startTime | date:'d' }}</b><span>{{ e.startTime | date:'MMM' }}</span></div>
              <span class="cat">{{ e.category }}</span>
            </div>
            <div class="body">
              <h3>{{ e.title }}</h3>
              <p class="muted">{{ e.venueName }}, {{ e.city }}</p>
              <p class="muted">{{ e.startTime | date:'EEE, h:mm a' }}</p>
              <div class="foot">
                <span>From <b>{{ e.basePrice | currency:'INR':'symbol':'1.0-0' }}</b></span>
                @if (e.status === 'CANCELLED') { <span class="chip cancelled">Cancelled</span> }
                @else if (e.soldSeats >= e.totalSeats) { <span class="chip cancelled">Sold out</span> }
                @else { <span class="left" [class.low]="pct(e) > 80">{{ e.totalSeats - e.soldSeats }} seats left</
span> }
              </div>
              <div class="bar" aria-hidden="true"><i [style.width.%]="pct(e)"></i></div>
            </div>
          </a>
        }
      </div>
    }
  </div>`,
  styles: [`
    .search { padding:10px 14px; border:1.5px solid var(--line); border-radius:99px; width:300px; max-width:100%;
 background:#fff; }
    .cats { display:flex; gap:8px; flex-wrap:wrap; margin-bottom:24px; }
    .cats button { border:1.5px solid var(--line); background:#fff; border-radius:99px; padding:6px 16px; cursor:
pointer; font-weight:500; }
    .cats button.on { background:var(--ink); color:#fff; border-color:var(--ink); }
    .grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(300px,1fr)); gap:22px; }
    .card { background:#fff; border-radius:var(--r-md); overflow:hidden; text-decoration:none; border:
1px solid var(--line); display:flex; flex-direction:column; transition:box-shadow .2s; }
    .card:hover { box-shadow:0 10px 28px #15132B22; }
    .poster { height:120px; position:relative; color:#fff; padding:14px; }
    .poster::after { content:''; position:absolute; left:0; right:0; bottom:-7px; height:14px; background:
radial-gradient(circle at 7px 0, var(--paper) 6px, transparent 7px) 0 0/22px 14px repeat-x; }
    .date { display:inline-flex; flex-direction:column; align-items:center; background:#ffffff26; border-radius:
var(--r-sm); padding:6px 12px; line-height:1; }
    .date b { font:800 1.7rem var(--head); } .date span { font-size:12.5px; margin-top:3px; }
    .cat { position:absolute; right:14px; top:14px; font-weight:600; font-size:13px; }
    .body { padding:20px 18px 16px; display:flex; flex-direction:column; gap:3px; flex:1; } .body h3 { margin-bottom:
4px; }
    .foot { display:flex; justify-content:space-between; align-items:center; margin-top:12px; }
    .left { font-size:13px; color:var(--ok); font-weight:600; } .left.low { color:var(--crimson); }
    .bar { height:4px; background:#E4E7F3; border-radius:4px; margin-top:8px; overflow:hidden; } .bar i { display:
block; height:100%; background:var(--ink-3); }`]
})
export class EventsComponent implements OnInit {
  private api = inject(ApiService);
  events = signal<EventItem[]>([]); loading = signal(true); error = signal('');
  q = ''; cat = ''; cats = CATEGORIES; color = catColor; private t?: any;
  ngOnInit() { this.fetch(); }
  load() { clearTimeout(this.t); this.t = setTimeout(() => this.fetch(), 250); }
  setCat(c: string) { this.cat = c; this.fetch(); }
  pct(e: EventItem) { return e.totalSeats ? Math.round(e.soldSeats * 100 / e.totalSeats) : 0; }
  private fetch() {
```

```
    this.loading.set(true);
    this.api.events(this.q, this.cat).subscribe({
      next: r => { this.events.set(r); this.loading.set(false); this.error.set(''); },
      error: e => { this.error.set(errorText(e)); this.loading.set(false); }
    });
  }
}
```

frontend/src/app/pages/login.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { AuthService } from '../core/auth.service';
import { errorText } from '../core/api.service';
@Component({
  selector: 'vv-login', standalone: true, imports: [FormsModule, RouterLink],
  template: `
  <div class="auth">
    <section class="stage" aria-hidden="true">
      <div class="curtain"></div>
      <div class="hall">
        @for (r of rows; track r; let ri = $index) {
          <div class="r">@for (s of r; track s) { <i [class.lit]="lit.has(ri + '-' + $index)" [style.animation-delay.
ms]="600 + (ri * 120)"></i> }</div>
        }
      </div>
      <h1>Every seat,<br>sold to someone<br>who'll be there.</h1>
    </section>
    <section class="form">
      <form (ngSubmit)="submit()" #f="ngForm">
        <h2>Welcome back</h2>
        <p class="muted">Sign in to book seats or run your events.</p>
        @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
        <div class="field"><label for="e">Email</
label><input id="e" name="email" type="email" [(ngModel)]="email" required autocomplete="email"></div>
        <div class="field"><label for="p">Password</
label><input id="p" name="password" type="password" [(ngModel)]="password" required autocomplete="current-password"></
div>
        <button class="btn block" [disabled]="f.invalid || busy()">{{ busy() ? 'Signing in…' : 'Sign in' }}</button>
        <p class="alt">New to VenueVista? <a routerLink="/register">Create an account</a></p>
        <div class="demo">
          <p class="hint">Demo accounts (click to fill)</p>
          <button type="button" (click)="fill('admin@venuevista.com','Admin@123')">Admin</button>
          <button type="button" (click)="fill('organizer@venuevista.com','Organizer@123')">Organiser</button>
          <button type="button" (click)="fill('customer@venuevista.com','Customer@123')">Customer</button>
        </div>
      </form>
    </section>
  </div>`,
  styles: [`
    .auth { display:grid; grid-template-columns:1.1fr 1fr; min-height:100vh; }
    .stage { background:var(--ink); color:#fff; padding:56px; display:flex; flex-direction:column; justify-content:
flex-end; gap:36px; position:relative; overflow:hidden; }
    .curtain { position:absolute; inset:0 0 auto 0; height:90px; background:linear-gradient(var(--crimson-d),
 transparent); opacity:.85; }
    .hall { display:flex; flex-direction:column; align-items:center; gap:8px; margin-bottom:auto; margin-top:80px; }
    .r { display:flex; gap:7px; } .r i { width:14px; height:14px; border-radius:4px 4px 7px 7px; background:
var(--ink-3); display:block; }
    .r i.lit { animation:light .5s forwards; } @keyframes light { to { background:var(--crimson); box-shadow:
0 0 14px var(--crimson); } }
    .stage h1 { font-size:clamp(2rem,3.4vw,3.2rem); }
    .form { display:grid; place-items:center; padding:32px; } form { width:100%; max-width:380px; }
    form h2 { font-size:2rem; margin-bottom:6px; } form .muted { margin-bottom:24px; }
    .alt { margin-top:18px; text-align:center; } .alt a { color:var(--crimson); font-weight:600; }
    .demo { margin-top:28px; padding-top:18px; border-top:1px solid var(--line); display:flex; gap:8px; align-items:
center; flex-wrap:wrap; }
    .demo .hint { width:100%; } .demo button { border:1.5px solid var(--line); background:#fff; border-radius:99px;
 padding:4px 14px; cursor:pointer; }
    .demo button:hover { border-color:var(--ink); }
    @media (max-width:820px) { .auth { grid-template-columns:1fr; } .stage { padding:28px; min-height:260px; } .
hall { display:none; } }`]
})
export class LoginComponent {
  private auth = inject(AuthService); private router = inject(Router);
  email = ''; password = ''; error = signal(''); busy = signal(false);
  rows = [6, 8, 10, 12, 14, 16].map(n => Array.from({ length: n }));
  lit = new Set(['1-3', '2-5', '2-6', '3-2', '3-9', '4-7', '5-4', '5-10', '5-11']);
  fill(e: string, p: string) { this.email = e; this.password = p; }
  submit() {
    this.busy.set(true); this.error.set('');
    this.auth.login({ email: this.email, password: this.password }).subscribe({
      next: () => this.router.navigate([this.auth.isStaff() ? '/dashboard' : '/events']),
      error: e => { this.error.set(errorText(e)); this.busy.set(false); }
    });
  }
}
```

frontend/src/app/pages/my-tickets.component.ts

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { ApiService, Order, errorText } from '../core/api.service';
@Component({
  selector: 'vv-my-tickets', standalone: true, imports: [DatePipe, CurrencyPipe, RouterLink],
  template: `
  <div class="page">
    <div class="page-head"><div><h1>My tickets</h1><p class="muted">Show the booking code at the door.</p></div></div>
    @if (error()) { <div class="alert">{{ error() }}</div> }
    @if (!orders().length && !loading()) { <div class="empty"><h3>No tickets yet</
h3><p>When you book seats they'll appear here.</p><a class="btn" routerLink="/events" style="margin-top:
14px">Find an event</a></div> }
    @for (o of orders(); track o.id) {
      <article class="ticket">
        <div class="main">
          <h2>{{ o.eventTitle }}</h2>
          <p>{{ o.startTime | date:'EEEE, d MMMM y · h:mm a' }}</p>
          <p class="muted">{{ o.venueName }}, {{ o.city }}</p>
          <div class="seats">@for (s of o.seats; track s.label) { <span class="seat-chip" [attr.data-tier]="s.
tier">{{ s.label }}</span> }</div>
        </div>
        <div class="stub">
          <div class="code" aria-hidden="true">@for (b of bars(o.code); track $index) { <i [style.width.px]="b"></
i> }</div>
          <b>{{ o.code }}</b>
          <span>{{ o.total | currency:'INR':'symbol':'1.0-0' }} · {{ o.seats.length }} seat{{ o.seats.length === 1 ?
 '' : 's' }}</span>
        </div>
      </article>
    }
  </div>`,
  styles: [`
    .ticket { display:grid; grid-template-columns:1fr 220px; background:#fff; border-radius:var(--r-md);
 margin-bottom:18px; border:1px solid var(--line); position:relative; }
    .main { padding:22px 26px; display:flex; flex-direction:column; gap:4px; } .main h2 { margin-bottom:6px; }
    .stub { border-left:2px dashed var(--line); padding:22px 18px; display:flex; flex-direction:column; align-items:
center; justify-content:center; gap:8px; text-align:center; font-size:13px; position:relative; }
    .stub::before,.stub::after { content:''; position:absolute; left:-11px; width:20px; height:20px; border-radius:
50%; background:var(--paper); border:1px solid var(--line); }
    .stub::before { top:-11px; } .stub::after { bottom:-11px; } .stub b { font:700 1.05rem var(--head);
 letter-spacing:.06em; }
    .code { display:flex; gap:2px; height:44px; align-items:stretch; } .code i { background:var(--ink); display:block;
 } .code i:nth-child(even) { background:transparent; }
    .seats { display:flex; gap:6px; flex-wrap:wrap; margin-top:12px; }
    .seat-chip { padding:3px 11px; border-radius:6px; color:#fff; font-weight:600; font-size:13px; }
    .seat-chip[data-tier=STANDARD] { background:var(--standard); } .seat-chip[data-tier=PREMIUM] { background:
var(--premium); } .seat-chip[data-tier=VIP] { background:var(--vip); }
    @media (max-width:640px) { .ticket { grid-template-columns:1fr; } .stub { border-left:0; border-top:
2px dashed var(--line); } .stub::before { left:-11px; top:-11px; } .stub::after { left:auto; right:-11px; top:-11px;
 bottom:auto; } }`]
})
export class MyTicketsComponent implements OnInit {
  private api = inject(ApiService);
  orders = signal<Order[]>([]); loading = signal(true); error = signal('');
  ngOnInit() { this.api.myOrders().subscribe({ next: o => { this.orders.set(o); this.loading.set(false); }, error:
 e => { this.error.set(errorText(e)); this.loading.set(false); } }); }
  bars(code: string) { return Array.from(code.replace('-', '')).flatMap(c => [1 + (c.charCodeAt(0) % 3), 1 + (c.
charCodeAt(0) % 2)]); }
}
```

frontend/src/app/pages/register.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { AuthService } from '../core/auth.service';
import { errorText } from '../core/api.service';
@Component({
  selector: 'vv-register', standalone: true, imports: [FormsModule, RouterLink],
  template: `
  <div class="wrap">
    <form class="panel" (ngSubmit)="submit()" #f="ngForm">
      <a routerLink="/login" class="back">Back to sign in</a>
      <h2>Create your account</h2>
      @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
      <div class="field"><label for="n">Full name</label><input id="n" name="name" [(ngModel)]="name" required></div>
      <div class="field"><label for="e">Email</
label><input id="e" name="email" type="email" [(ngModel)]="email" required></div>
      <div class="field"><label for="p">Password</
label><input id="p" name="password" type="password" [(ngModel)]="password" required minlength="8">
        <span class="hint">At least 8 characters.</span></div>
      <fieldset>
        <legend>What will you do on VenueVista?</legend>
        <label class="opt" [class.
on]="role==='CUSTOMER'"><input type="radio" name="role" value="CUSTOMER" [(ngModel)]="role"><strong>Attend events</
strong><span>Browse, pick seats and keep your tickets.</span></label>
        <label class="opt" [class.
on]="role==='ORGANIZER'"><input type="radio" name="role" value="ORGANIZER" [(ngModel)]="role"><strong>Organise events<
/strong><span>Manage venues, seating, pricing and sales.</span></label>
      </fieldset>
      <button class="btn block" [disabled]="f.invalid || busy()">{{ busy() ? 'Creating account…' :
 'Create account' }}</button>
    </form>
  </div>`,
  styles: [`
    .wrap { min-height:100vh; display:grid; place-items:center; padding:24px; } form { width:100%; max-width:460px;
 padding:32px; }
    .back { font-size:13px; color:var(--muted); display:inline-block; margin-bottom:14px; } h2 { margin-bottom:20px; }
    fieldset { border:0; padding:0; margin:0 0 20px; display:grid; gap:10px; } legend { font-weight:600; font-size:13.
5px; margin-bottom:8px; padding:0; }
    .opt { display:grid; gap:2px; border:1.5px solid var(--line); border-radius:var(--r-md); padding:12px 14px;
 cursor:pointer; }
    .opt input { position:absolute; opacity:0; } .opt span { color:var(--muted); font-size:13.5px; }
    .opt.on { border-color:var(--crimson); background:#FDF3F5; } .opt:focus-within { outline:
3px solid var(--periwinkle); }`]
})
export class RegisterComponent {
  private auth = inject(AuthService); private router = inject(Router);
  name = ''; email = ''; password = ''; role = 'CUSTOMER'; error = signal(''); busy = signal(false);
  submit() {
    this.busy.set(true); this.error.set('');
    this.auth.register({ name: this.name, email: this.email, password: this.password, role: this.role }).subscribe({
      next: () => this.router.navigate([this.auth.isStaff() ? '/dashboard' : '/events']),
      error: e => { this.error.set(errorText(e)); this.busy.set(false); }
    });
  }
}
```

frontend/src/app/pages/shell.component.ts

```
import { Component, inject } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { AuthService } from '../core/auth.service';
@Component({
  selector: 'vv-shell', standalone: true, imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
  <div class="layout">
    <aside>
      <a routerLink="/events" class="brand"><span class="mark">V</span><span>VenueVista</span></a>
      <nav aria-label="Main">
        <a routerLink="/events" routerLinkActive="on" [routerLinkActiveOptions]="{exact:true}">Discover events</a>
        @if (auth.user()) { <a routerLink="/tickets" routerLinkActive="on">My tickets</a> }
        @if (auth.isStaff()) {
          <p class="group">Organiser tools</p>
          <a routerLink="/dashboard" routerLinkActive="on">Sales analytics</a>
          <a routerLink="/events/new" routerLinkActive="on">Create event</a>
          <a routerLink="/venues" routerLinkActive="on">Venues &amp;amp; seating</a>
        }
      </nav>
      <div class="me">
        @if (auth.user(); as u) {
          <div class="avatar">{{ u.name.charAt(0) }}</div>
          <div class="who"><strong>{{ u.name }}</strong><small>{{ u.role.toLowerCase() }}</small></div>
          <button class="out" (click)="auth.logout()" aria-label="Sign out">Sign out</button>
        } @else { <a routerLink="/login" class="btn block">Sign in</a> }
      </div>
    </aside>
    <main><router-outlet /></main>
  </div>`,
  styles: [`
    .layout { display:grid; grid-template-columns:248px 1fr; min-height:100vh; }
    aside { background:var(--ink); color:#E9E9F6; padding:22px 16px; display:flex; flex-direction:column; position:
sticky; top:0; height:100vh; }
    .brand { display:flex; align-items:center; gap:10px; font:800 1.3rem var(--head); text-decoration:none; color:
#fff; margin-bottom:28px; padding:0 8px; }
    .mark { width:34px; height:34px; border-radius:9px; background:var(--crimson); display:grid; place-items:center;
 font-size:1.2rem; }
    nav { display:flex; flex-direction:column; gap:2px; flex:1; }
    nav a { color:#C9CAE6; text-decoration:none; padding:10px 12px; border-radius:var(--r-sm); font-weight:500; }
    nav a:hover { background:var(--ink-2); color:#fff; } nav a.on { background:var(--ink-3); color:#fff; box-shadow:
inset 3px 0 0 var(--crimson); }
    .group { margin:20px 12px 6px; font-size:12.5px; color:#8E90B8; }
    .me { display:flex; align-items:center; gap:10px; padding:12px 8px; border-top:1px solid var(--ink-3); flex-wrap:
wrap; }
    .avatar { width:36px; height:36px; border-radius:50%; background:var(--brass); color:#fff; display:grid;
 place-items:center; font-weight:700; }
    .who { display:flex; flex-direction:column; line-height:1.2; flex:1; } .who small { color:#9A9CC4; text-transform:
capitalize; }
    .out { background:none; border:1px solid var(--ink-3); color:#C9CAE6; border-radius:var(--r-sm); padding:5px 10px;
 cursor:pointer; font-size:12.5px; }
    .out:hover { background:var(--ink-2); }
    @media (max-width: 860px) {
      .layout { grid-template-columns:1fr; } aside { position:static; height:auto; flex-direction:row; flex-wrap:wrap;
 align-items:center; gap:8px; }
      .brand { margin:0; } nav { flex-direction:row; flex-wrap:wrap; } .group { display:none; } .me { border:0; }
    }`]
})
export class ShellComponent { auth = inject(AuthService); }
```

frontend/src/app/pages/venues.component.ts

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ApiService, Venue, errorText } from '../core/api.service';
@Component({
  selector: 'vv-venues', standalone: true, imports: [FormsModule],
  template: `
  <div class="page">
    <div class="page-head"><div><h1>Venues &amp; seating</h1><p class="muted">Design the seating chart once,
 reuse it for every event.</p></div></div>
    <div class="layout">
      <form class="panel" (ngSubmit)="add()" #f="ngForm">
        <h2>Add a venue</h2>
        @if (error()) { <div class="alert" role="alert">{{ error() }}</div> }
        <div class="field"><label for="n">Venue name</label><input id="n" name="n" [(ngModel)]="m.name" required></
div>
        <div class="field"><label for="c">City</label><input id="c" name="c" [(ngModel)]="m.city" required></div>
        <div class="field"><label for="a">Address</label><input id="a" name="a" [(ngModel)]="m.address"></div>
        <div class="two">
          <div class="field"><label for="r">Rows (A–Z)</
label><input id="r" name="r" type="number" min="1" max="26" [(ngModel)]="m.rowsCount" required></div>
          <div class="field"><label for="s">Seats per row</
label><input id="s" name="s" type="number" min="1" max="40" [(ngModel)]="m.seatsPerRow" required></div>
        </div>
        <div class="preview" aria-label="Seating preview"><div class="st">STAGE</div>
          @for (r of preview(); track $index) { <div class="pr">@for (s of seatsArr(); track $index) { <i [attr.
data-t]="tierOf($index, r)"></i> }</div> }
        </div>
        <p class="hint">{{ (m.rowsCount||0) * (m.seatsPerRow||0) }} seats. Front 20% of rows are VIP,
 next 40% Premium.</p>
        <button class="btn block" [disabled]="f.invalid || busy()">Save venue</button>
      </form>
      <section>
        @if (!venues().length) { <div class="empty"><h3>No venues yet</
h3><p>Add your first venue to start creating events.</p></div> }
        @for (v of venues(); track v.id) {
          <article class="panel venue">
            <div><h3>{{ v.name }}</h3><p class="muted">{{ v.address }}{{ v.address ? ', ' : '' }}{{ v.city }}</p>
              <p><b>{{ v.capacity }}</b> seats · {{ v.rowsCount }} rows × {{ v.seatsPerRow }}</p></div>
            <button class="btn danger sm" (click)="remove(v)">Remove</button>
          </article>
        }
      </section>
    </div>
  </div>`,
  styles: [`
    .layout { display:grid; grid-template-columns:380px 1fr; gap:24px; align-items:start; } .two { display:grid;
 grid-template-columns:1fr 1fr; gap:12px; }
    form h2 { margin-bottom:14px; } .venue { display:flex; justify-content:space-between; gap:12px; margin-bottom:
14px; }
    .preview { background:var(--paper); border-radius:var(--r-sm); padding:12px; margin-bottom:8px; overflow:auto;
 display:flex; flex-direction:column; align-items:center; gap:3px; }
    .st { background:var(--ink); color:#fff; font:700 10px var(--head); letter-spacing:.2em; padding:3px 30px;
 border-radius:3px; margin-bottom:6px; }
    .pr { display:flex; gap:3px; } .pr i { width:8px; height:8px; border-radius:2px 2px 4px 4px; display:block; }
    i[data-t=V] { background:var(--vip); } i[data-t=P] { background:var(--premium); } i[data-t=S] { background:
var(--standard); }
    @media (max-width:900px) { .layout { grid-template-columns:1fr; } }`]
})
export class VenuesComponent implements OnInit {
  private api = inject(ApiService);
  venues = signal<Venue[]>([]); error = signal(''); busy = signal(false);
  m: any = { name: '', city: '', address: '', rowsCount: 10, seatsPerRow: 16 };
  ngOnInit() { this.load(); }
  load() { this.api.venues().subscribe(v => this.venues.set(v)); }
  preview() { return Array.from({ length: Math.min(Math.max(+this.m.rowsCount || 0, 0), 26) }, (_, i) => i); }
  seatsArr() { return Array.from({ length: Math.min(Math.max(+this.m.seatsPerRow || 0, 0), 40) }); }
  tierOf(_: number, row: number) { const n = +this.m.rowsCount; return row < Math.ceil(n * .2) ? 'V' : row < Math.
ceil(n * .6) ? 'P' : 'S'; }
  add() {
    this.busy.set(true); this.error.set('');
    this.api.createVenue(this.m).subscribe({
      next: () => { this.busy.set(false); this.m = { name: '', city: '', address: '', rowsCount: 10, seatsPerRow:
 16 }; this.load(); },
      error: e => { this.error.set(errorText(e)); this.busy.set(false); }
    });
  }
  remove(v: Venue) {
    if (!confirm(`Remove ${v.name}?`)) return;
    this.api.deleteVenue(v.id).subscribe({ next: () => this.load(), error: e => this.error.set(errorText(e)) });
  }
}
```

frontend/src/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>VenueVista · EventPro Ticketing</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12..96,500;12..96,700;12..96,800&
family=Instrument+Sans:wght@400;500;600&display=swap" rel="stylesheet">
</head>
<body><app-root></app-root></body>
</html>
```

frontend/src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';
bootstrapApplication(AppComponent, appConfig).catch(err => console.error(err));
```

frontend/src/styles.scss

```
:root {
  --ink:#15132B; --ink-2:#262347; --ink-3:#3A3768; --paper:#EEF0F7; --card:#fff; --line:#D9DCEA; --muted:#5E617F;
  --crimson:#C4122F; --crimson-d:#9E0E26; --brass:#B8862B; --periwinkle:#5F6FD8; --ok:#1F7A4D;
  --standard:var(--periwinkle); --premium:var(--brass); --vip:var(--crimson);
  --r-sm:6px; --r-md:12px; --r-lg:20px;
  --head:'Bricolage Grotesque', 'Segoe UI', sans-serif; --body:'Instrument Sans', 'Segoe UI', sans-serif;
}
* { box-sizing:border-box; }
html,body { margin:0; background:var(--paper); color:var(--ink); font:400 15px/1.55 var(--body); }
h1,h2,h3,h4 { font-family:var(--head); letter-spacing:-.02em; line-height:1.1; margin:0; }
h1 { font-size:2.4rem; font-weight:800; } h2 { font-size:1.5rem; font-weight:700; } h3 { font-size:1.1rem;
 font-weight:700; }
p { margin:0; } a { color:inherit; }
button, input, select, textarea { font:inherit; color:inherit; }
:focus-visible { outline:3px solid var(--periwinkle); outline-offset:2px; }
.muted { color:var(--muted); } .row { display:flex; gap:12px; align-items:center; } .grow { flex:1; }
.page { max-width:1180px; margin:0 auto; padding:32px 28px 64px; }
.page-head { display:flex; justify-content:space-between; align-items:flex-end; gap:16px; margin-bottom:24px;
 flex-wrap:wrap; }

.btn { display:inline-flex; align-items:center; justify-content:center; gap:8px; border:0; cursor:pointer; padding:
10px 18px;
  border-radius:var(--r-sm); background:var(--crimson); color:#fff; font-weight:600; text-decoration:none; transition:
background .15s, transform .1s; }
.btn:hover { background:var(--crimson-d); } .btn:active { transform:translateY(1px); }
.btn:disabled { background:#B9BBD0; cursor:not-allowed; }
.btn.ghost { background:transparent; color:var(--ink); box-shadow:inset 0 0 0 1.5px var(--line); }
.btn.ghost:hover { background:#fff; } .btn.danger { background:transparent; color:var(--crimson); box-shadow:
inset 0 0 0 1.5px var(--crimson); }
.btn.sm { padding:6px 12px; font-size:13px; } .btn.block { width:100%; }

.field { display:flex; flex-direction:column; gap:6px; margin-bottom:16px; }
.field label { font-weight:600; font-size:13.5px; }
.field input, .field select, .field textarea { padding:11px 12px; border:1.5px solid var(--line); border-radius:
var(--r-sm); background:#fff; }
.field input:focus, .field select:focus, .field textarea:focus { border-color:var(--periwinkle); outline:none;
 box-shadow:0 0 0 3px #5F6FD833; }
.hint { font-size:12.5px; color:var(--muted); }
.alert { padding:10px 14px; border-radius:var(--r-sm); font-size:14px; margin-bottom:16px; background:#FBE7EA; color:
var(--crimson-d); border-left:4px solid var(--crimson); }
.alert.ok { background:#E3F4EB; color:var(--ok); border-color:var(--ok); }
.panel { background:var(--card); border-radius:var(--r-md); padding:22px; border:1px solid var(--line); }
.empty { text-align:center; padding:56px 20px; color:var(--muted); } .empty h3 { color:var(--ink); margin-bottom:6px;
 }
.chip { display:inline-block; padding:2px 10px; border-radius:99px; font-size:12.5px; font-weight:600; background:
#E6E8F5; color:var(--ink-3); }
.chip.cancelled { background:#FBE7EA; color:var(--crimson-d); }
.tier-dot { width:10px; height:10px; border-radius:50%; display:inline-block; margin-right:6px; }
.tier-dot[data-tier=STANDARD] { background:var(--standard); } .tier-dot[data-tier=PREMIUM] { background:
var(--premium); } .tier-dot[data-tier=VIP] { background:var(--vip); }
.skeleton { background:linear-gradient(90deg,#E4E7F3,#F3F4FA,#E4E7F3); background-size:200% 100%; animation:sk 1.
2s infinite; border-radius:var(--r-md); }
@keyframes sk { to { background-position:-200% 0; } }
@media (prefers-reduced-motion: reduce) { * { animation:none !important; transition:none !important; } }
```

frontend/tsconfig.app.json

```
{ "extends": "./tsconfig.json", "compilerOptions": { "outDir": "./out-tsc/app", "types": [] },
  "files": ["src/main.ts"], "include": ["src/**/*.d.ts"] }
```

frontend/tsconfig.json

```
{ "compileOnSave": false,
  "compilerOptions": { "outDir": "./dist/out-tsc", "strict": true, "noImplicitOverride": true, "skipLibCheck": true,
    "esModuleInterop": true, "sourceMap": true, "declaration": false, "experimentalDecorators": true,
    "moduleResolution": "node", "importHelpers": true, "target": "ES2022", "module": "ES2022",
 "useDefineForClassFields": false,
    "lib": ["ES2022", "dom"] },
  "angularCompilerOptions": { "strictInjectionParameters": true, "strictInputAccessModifiers": true,
 "strictTemplates": true } }
```

README.md

```
# VenueVista — EventPro Ticketing & Venue Management

Event ticketing and venue management with seating charts, ticket sales and sales analytics.
**Stack:** Java 17 · Spring Boot 3 (Security + JWT, JPA) · Angular 17 (standalone, signals) · Oracle DB

## Features
| Area | What it does |
|---|---|
| Auth & roles | Register / sign in (JWT). Roles: ADMIN, ORGANIZER, CUSTOMER with guarded routes and API rules |
| Discover events | Search by title/venue/city, category filter, seats-left meter, sold-out / cancelled states |
| Seating chart | Interactive seat map per event, tiered pricing (Standard / Premium / VIP), aisle gap,
 keyboard + screen-reader labels |
| Ticket sales | Pick up to 8 seats, atomic checkout with row-level locking (no double selling), booking code |
| My tickets | Ticket-stub view with tier-coloured seats and barcode-style code |
| Venue management | Define rows × seats per venue with live seating preview; remove unused venues |
| Event management | Create events (seats auto-generated from venue layout and priced by tier), cancel events |
| Sales analytics | Revenue, seat fill %, average order, 14-day revenue trend, tier mix donut, top events |

## 1. Database (Oracle or local H2)
For Oracle,
 create a local environment file from the example and set the database account and a private JWT signing secret:
```bash
cp .env.example .env
```
Edit `.env` with your own values. It is ignored by Git.
 The backend loads it when started from the project root or `backend` directory.
 Keep the same Oracle username and password you enter in the setup script.

```sql
-- as SYSTEM, in your PDB (e.g. XEPDB1)  → database/01_create_user.sql
-- The script securely prompts for the new account password.
```
Tables are created automatically by Hibernate on first run. Set `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD` in `.
env` for your Oracle installation.

For a local demo without Oracle, use the `local` Spring profile.
 It starts an in-memory H2 database in Oracle compatibility mode and reseeds demo data each time the backend starts.

2. Backend
```bash
cd backend && mvn spring-boot:run      # Oracle, http://localhost:8081
# or, without Oracle:
mvn spring-boot:run -Dspring-boot.run.profiles=local
```
On first start demo data is seeded (3 venues, 6 events, 2 weeks of sales).

3. Frontend
```bash
cd frontend && npm install && npm start   # http://localhost:4200
```

Demo accounts
Role	Email	Password
Admin	admin@venuevista.com	Admin@123
Organiser	organizer@venuevista.com	Organizer@123
Customer	customer@venuevista.com	Customer@123

API overview
`POST /api/auth/register|login` · `GET /api/events?q=&category=` · `GET /api/events/{id}/seats` · `POST /api/
events` · `DELETE /api/events/{id}` (cancel) ·
`GET|POST /api/venues` · `DELETE /api/venues/{id}` · `POST /api/orders` · `GET /api/orders/mine` · `GET /api/
analytics/summary`

Before production
Set a unique `JWT_SECRET`, DB credentials, and CORS origin in `.env`; set `ddl-auto:
 validate` with managed migrations; add payment-gateway integration at the order step. Never commit `.
env` or paste real credentials into source files.
```